

ResumeAI Backend – API

Documentation for Frontend Integration

Base URL: <http://127.0.0.1:8000> (local) or your deployment URL

API Version: 1.0

Table of Contents

1. Authentication
 2. Headers
 3. Error Handling
 4. Auth APIs
 5. Profile APIs
 6. Resume APIs
 7. Dashboard APIs
 8. Templates API
 9. Pricing APIs
 10. Settings APIs
 11. Help Center APIs
 12. Juno AI Assistant APIs
 13. Data Models
 14. Integration Workflows
 15. Swagger & ReDoc
-

1. Authentication

All endpoints except **Signup**, **Login**, **Refresh**, **Logout**, and **Pricing Plans List** require authentication.

Token Usage

- **Access Token:** Include in the [Authorization](#) header for every protected request.
- **Refresh Token:** Use to get a new access token when it expires (e.g. backend returns [401](#)).
- If your HTTP client automatically adds [Bearer](#), do not add it manually. Otherwise, include [Bearer](#).

Authorization Header

Authorization: Bearer <access_token>

Token Refresh Flow (Frontend Standard)

When any protected API returns **401 Unauthorized**:

1. Call **POST /auth/refresh** with the refresh token.
2. Store the new **access_token** and **refresh_token**.
3. Retry the original request with the new access token.
4. If refresh fails (401), clear auth state and redirect to login.

2. Headers

Header	Required	Description
Content-Type	Yes (for JSON bodies)	application/json
Authorization	Yes (protected routes)	Bearer <access_token>

Uploads: Use **multipart/form-data**.

Do **not** set **Content-Type** manually for uploads; the browser/client will add the correct boundary.

3. Error Handling

Standard Error Response

```
{
  "detail": "Error message string"
}
```

Validation Error Response (422)

```
{
  "detail": [
    {
      "loc": ["body", "email"],
      "msg": "value is not a valid email address",
      "type": "value_error.email"
    }
  ]
}
```

```
}  
]  
}
```

HTTP Status Codes

Code	Meaning
200	Success
201	Created
204	No Content (success, empty body)
400	Bad Request – invalid input
401	Unauthorized – missing/invalid token
403	Forbidden – insufficient permissions
404	Not Found
422	Unprocessable Entity – validation failed

4. Auth APIs

4.1 Signup

Endpoint: `POST /auth/signup`

Auth: Not required

Request Body

```
{  
  "name": "Alex Doe",  
  "email": "alex@example.com",  
  "password": "SecurePass123!"  
}
```

Response (200)

```
{  
  "message": "User registered successfully",  
  "user_id": 1  
}
```

Errors

- 400 – Email already registered
-

4.2 Login

Endpoint: POST /auth/login

Auth: Not required

Request Body

```
{
  "email": "alex@example.com",
  "password": "SecurePass123!"
}
```

Response (200)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

Frontend Notes

- Store `access_token` and `refresh_token`.
- Use `access_token` for all protected endpoints.

Errors

- 401 – Invalid email or password
-

4.3 Refresh Tokens

Endpoint: POST /auth/refresh

Auth: Not required

Request Body

```
{
  "refresh_token": "<REFRESH_TOKEN>"
}
```

Response (200)

Same shape as Login.

Errors

- 401 – Invalid or revoked refresh token
-

4.4 Logout All Sessions

Endpoint: POST /auth/logout-all

Auth: Required

Request Body: None

Response (200)

```
{
  "message": "Logged out from all sessions"
}
```

5. Profile APIs

5.1 Get Current User

Endpoint: GET /profile/me

Auth: Required

Response (200)

```
{
  "id": "1",
  "name": "Alex Doe",
  "email": "alex@example.com",
  "role": "user",
  "phone": "+1 555 555 5555",
  "location": "New York, NY",
  "linkedin_url": "https://linkedin.com/in/alex",
  "portfolio_url": "https://alex.dev",
  "credits_remaining": 150
}
```

5.2 Update Profile

Endpoint: `PATCH /profile/me`

Auth: Required

Request Body (all optional)

```
{
  "name": "Alex Doe",
  "phone": "+1 555 555 5555",
  "location": "New York, NY",
  "linkedin_url": "https://linkedin.com/in/alex",
  "portfolio_url": "https://alex.dev"
}
```

Response (200)

Same shape as Get Current User (updated profile).

5.3 Change Password

Endpoint: `POST /profile/change-password`

Auth: Required

Request Body

```
{
  "old_password": "SecurePass123!",
  "new_password": "NewSecurePass456!",
  "confirm_password": "NewSecurePass456!"
}
```

Response (200)

Same shape as Get Current User.

Errors

- `400` – "Current password is incorrect"
 - `400` – "New password and confirmation do not match"
 - `400` – "New password must be at least 6 characters"
-

5.4 Sync Profile from Resume

Endpoint: `POST /profile/sync-from-resume/{resume_id}`

Auth: Required

Path Parameters

- `resume_id` – per-user resume ID (e.g. 1)

Response (200)

Same shape as Get Current User.

Errors

- `404` – Resume not found
-

6. Resume APIs

Resume IDs are **per-user** (1, 2, 3, ...), not global.

6.1 Create Resume (Scratch)

Endpoint: `POST /resumes?mode=scratch`

Auth: Required

Query Params

- `mode` – `scratch` (default) or `empty`

Request Body

```
{
  "title": "My Resume",
  "template_id": null,
  "content": null
}
```

Response (201)

```
{
  "id": 1,
  "title": "My Resume",
  "template_id": null,
  "status": "draft",
  "content": { },
  "created_at": "2025-02-01T12:00:00Z",
  "updated_at": "2025-02-01T12:00:00Z"
}
```

}

6.2 Create Resume from Upload

Endpoint: `POST /resumes/from-upload`

Auth: Required

Content-Type: `multipart/form-data`

Query Params (optional)

- `title`
- `template_id`

Form Data

Key	Type	Required	Description
<code>file</code>	File	Yes	PDF or DOCX (max 10MB)

Response (201)

Same structure as Create Resume (Scratch), with `content` populated.

Errors

- `400` – Unsupported format / file too large / no filename
- `422` – Parse error

Frontend Example (Fetch)

```
const formData = new FormData();
formData.append("file", file);

const res = await fetch(`${baseUrl}/resumes/from-upload`, {
  method: "POST",
  headers: { Authorization: `Bearer ${accessToken}` },
  body: formData,
});
```

6.3 Merge Upload into Existing Resume

Endpoint: `PATCH /resumes/{resume_id}/from-upload`

Auth: Required

Content-Type: `multipart/form-data`

Form Data

- `file` – PDF or DOCX

Response (200)

Updated resume object.

6.4 Parse Upload (Preview Only)

Endpoint: `POST /resumes/parse-upload`

Auth: Required

Content-Type: `multipart/form-data`

Form Data

- `file` – PDF or DOCX

Response (200) (content only, no DB save)

```
{
  "info": {
    "full_name": "Ali Shayan Amir",
    "email": "a@example.com",
    "phone": "+9233622247",
    "location": null,
    "linkedin_url": "https://linkedin.com/in/...",
    "portfolio_url": null
  },
  "experience": [],
  "education": [],
  "skills": ["Python", "React"],
  "summary": "...",
  "job_description": {},
  "custom": {}
}
```

6.5 List Resumes

Endpoint: `GET /resumes`

Auth: Required

Query Params

Param	Type	Default	Description
<code>limit</code>	int	50	Max items (1–100)
<code>offset</code>	int	0	Pagination offset
<code>q</code>	string	-	Search in title

Response (200)

```
{
  "items": [
    {
      "id": 1,
      "title": "My Resume",
      "updated_at": "2025-02-01T12:00:00Z",
      "status": "draft"
    }
  ],
  "total": 5
}
```

6.6 Get Resume

Endpoint: `GET /resumes/{resume_id}`

Auth: Required

Response (200)

```
{
  "id": 1,
  "title": "My Resume",
  "template_id": null,
  "status": "draft",
  "content": {
    "info": {},
    "experience": [],
    "education": [],
    "skills": [],
    "summary": "",
    "job_description": {},
    "custom": {}
  },
}
```

```
"created_at": "2025-02-01T12:00:00Z",
"updated_at": "2025-02-01T12:00:00Z"
}
```

6.7 Update Resume

Endpoint: `PATCH /resumes/{resume_id}`

Auth: Required

Request Body (all optional)

```
{
  "title": "Updated Title",
  "template_id": 1,
  "status": "draft",
  "content": {}
}
```

Response (200)

Full resume object.

6.8 Duplicate Resume

Endpoint: `POST /resumes/{resume_id}/duplicate`

Auth: Required

Response (201)

New resume object (title includes "(Copy)").

6.9 Delete Resume

Endpoint: `DELETE /resumes/{resume_id}`

Auth: Required

Response

- `204 No Content`
-

6.10 Get Content (Single Section)

Endpoint: `GET /resumes/{resume_id}/content?section={section}`
Auth: Required

Query Param

- `section` – `info | experience | education | skills | summary | job_description | custom`

Response (200)
Returns the section content (object/array/string depending on section).

6.11 Patch Content (Single Section)

Endpoint: `PATCH /resumes/{resume_id}/content?section={section}`
Auth: Required

Request Body
Varies by section.

Response (200)

```
{
  "message": "Updated",
  "section": "info",
  "content": {}
}
```

Section Payload Examples

Section	Body Type	Example
<code>info</code>	object	<code>{"full_name": "Alex", "email": "a@x.com", "phone": "+1"}</code>
<code>experience</code>	array	<code>[{"role": "SE", "company": "Acme", "start_date": "2020", "end_date": "2024", "description": "..."}]</code>
<code>education</code>	array	<code>[{"school": "MIT", "degree": "BS", "start_date": "2016", "end_date": "2020"}]</code>
<code>skills</code>	array	<code>["Python", "React", "SQL"]</code>

```
summary      stri  "Experienced software engineer..."
ng

job_desc     obj    {"job_title":"SE","company":"X","description":"..."}
ription      ect

custom       obj    {"anything": "allowed"}
ect
```

6.12 Save ATS Score

Endpoint: `POST /resumes/{resume_id}/ats-score`

Auth: Required

Request Body

```
{
  "overall_score": 86,
  "max_score": 100,
  "category_scores": {
    "keyword_match": {
      "label": "Keyword Match",
      "description": "Matched 41/50",
      "score": 41,
      "max": 50
    }
  },
  "recommendations": [
    "Add keyword \"Agile\" in Experience section",
    "Use consistent date format"
  ]
}
```

Response (201)

ATS score object including fields like `id`, `resume_id`, and timestamps.

6.13 Get ATS Score

Endpoint: `GET /resumes/{resume_id}/ats-score`

Auth: Required

Response (200)

ATS score object or `null` if none exists.

6.14 Get Section Definitions (Dynamic Forms)

Endpoint: `GET /resumes/sections/definition?section={section}`

Auth: Required

Response (200)

```
{
  "section": "info",
  "fields": [
    { "key": "full_name", "label": "Full Name", "type": "text", "required": true },
    { "key": "email", "label": "Email", "type": "email", "required": true }
  ]
}
```

7. Dashboard APIs

7.1 Summary

Endpoint: `GET /dashboard/summary`

Auth: Required

Response (200)

```
{
  "welcome_name": "Alex",
  "resume_count": 5,
  "credits_remaining": 150,
  "recent": [
    { "id": 1, "title": "My Resume", "updated_at": "2025-02-01T12:00:00Z" }
  ],
  "suggested_templates": [
    { "id": 1, "name": "Modern", "slug": "modern", "preview_url": "...", "is_premium": false }
  ]
}
```

7.2 Recent Activity

Endpoint: `GET /dashboard/recent-activity?limit=10`

Auth: Required

Response (200)

Array of `{id, title, updated_at}`.

8. Templates API

8.1 List Templates

Endpoint: `GET /templates`

Auth: Required

Query Params

- `q, style, industry, limit` (default 12), `offset`

Response (200) (list item example)

```
{
  "id": 1,
  "name": "Modern",
  "slug": "modern",
  "preview_url": "...",
  "is_premium": false,
  "style": "Modern",
  "industry": null
}
```

9. Pricing APIs

9.1 List Plans (Public)

Endpoint: `GET /pricing/plans`

Auth: Not required

Query Params

- `active_only` (default true)

Response (200) (plan example)

```
{
  "id": 1,
  "name": "Free",
  "slug": "free",

```

```
"label": "Head Start",
"price": 0,
"credits": 5,
"description": "Perfect for getting started.",
"features": ["5 AI Credits", "Basic Templates", "Standard Support"],
"is_popular": false,
"display_order": 1
}
```

9.2 Choose Plan

Endpoint: `POST /pricing/plans/{plan_id}/choose`

Auth: Required

Response (200)

```
{
  "message": "Plan 'Starter' selected",
  "credits": 50
}
```

10. Settings APIs

10.1 Get Preferences

Endpoint: `GET /settings/preferences`

Auth: Required

Response (200)

```
{
  "dark_mode": true,
  "accent_color": "blue",
  "email_notifications": true,
  "in_app_notifications": true,
  "two_factor_enabled": false
}
```

10.2 Update Preferences

Endpoint: `PATCH /settings/preferences`

Auth: Required

Request Body (all optional)

```
{
  "dark_mode": true,
  "accent_color": "blue",
  "email_notifications": true,
  "in_app_notifications": true,
  "two_factor_enabled": false
}
```

Response (200)

Updated preferences object.

10.3 Account Summary

Endpoint: `GET /settings/account/summary`

Auth: Required

Response (200)

```
{
  "current_plan": "Pro",
  "credits_remaining": 540
}
```

10.4 Export Data

Endpoint: `GET /settings/account/export`

Auth: Required

Response (200)

JSON file download (via `Content-Disposition: attachment`).

10.5 Delete Account

Endpoint: `DELETE /settings/account`

Auth: Required

Response

- 204 No Content
-

11. Help Center APIs

11.1 List Topics

Endpoint: GET /help/topics

Auth: Required

Response (200)

Array of {id, name, slug, description, icon, display_order}.

11.2 Search Articles

Endpoint: GET /help/articles

Auth: Required

Query Params

- q, topic_id, featured_only, faq_only, limit, offset

Response (200)

Array of article list items.

11.3 Featured Articles

Endpoint: GET /help/articles/featured?limit=10

Auth: Required

11.4 FAQs

Endpoint: GET /help/articles/faqs?limit=20

Auth: Required

11.5 Article Detail by Slug

Endpoint: GET /help/articles/{slug}

Auth: Required

Response (200)

Full article with `content`, `topic_name`, etc.

12. Juno AI Assistant APIs

12.1 List Example Prompts

Endpoint: GET /juno/prompts

Auth: Required

Query Params

- `category` (optional filter)

Response (200)

Array of `{id, text, category, display_order}`.

13. Data Models

Resume Content Structure (TypeScript)

```
interface ResumeContent {
  info: {
    full_name: string;
    email: string | null;
    phone: string;
    location: string | null;
    linkedin_url: string | null;
    portfolio_url: string | null;
  };
  experience: Array<{
    role?: string;
    company?: string;
    start_date?: string;
    end_date?: string;
    location?: string;
    description?: string;
  }>;
  education: Array<{
    school: string;
```

```
degree: string;
start_date: string;
end_date: string;
location?: string;
field_of_study?: string;
}>;
skills: string[];
summary: string;
job_description: {
  job_title: string;
  company: string;
  location?: string;
  description: string;
};
custom: Record<string, unknown>;
}
```

Resume Sections (for PATCH content)

- `info` – object
 - `experience` – array
 - `education` – array
 - `skills` – array
 - `summary` – string
 - `job_description` – object
 - `custom` – object
-

14. Integration Workflows

App Startup

1. Check for stored `access_token`.
 2. If present, call `GET /profile/me`.
 3. If `401`, call `POST /auth/refresh`.
 4. If refresh fails `401`, redirect to login.
-

Create New Resume (Scratch)

1. `POST /resumes?mode=scratch` with `{ "title": "My Resume" }`
2. Use returned `id` for edit screen.

3. Update with `PATCH /resumes/{id}/content?section=...`
-

Create Resume from Upload

1. User selects PDF/DOCX.
 2. Optional: `POST /resumes/parse-upload` for preview.
 3. `POST /resumes/from-upload` with form-data `file`.
 4. Use returned resume directly (no extra fetch required).
-

Merge Upload into Existing Resume

1. User opens resume (`resume_id`)
 2. Upload PDF/DOCX
 3. `PATCH /resumes/{resume_id}/from-upload`
 4. Response is updated resume with merged content
-

“My Resumes” Page

1. `GET /resumes?limit=50&offset=0`
 2. Render `items`
 3. Use `total` for pagination
 4. Link with per-user `id`
-

Change Password Flow

1. Collect old/new/confirm
 2. Call `POST /profile/change-password`
 3. Show success or error from `detail`
-

15. Swagger & ReDoc

- Swagger UI: `{base_url}/docs`
- ReDoc: `{base_url}/redoc`

Both list all endpoints and allow testing.

